

CM52065 Natural Language Processing

Support Ticket Classification

1 Introduction

This report investigates the classification of IT support tickets using Natural Language Processing. NLP is well suited for this task as it can interpret different elements of text, including morphology, grammar, syntax, and semantics ([Uber AI, 2018](#)).

The study uses 308 support tickets belonging to 14 different categories including shipping delay, payment issue, billing history, etc. Two different pre-processing pipelines are explored for feature extraction from the raw text with a single classification algorithm for each. The report summarises the pre-processing pipelines, modelling approach, a comparative evaluation of their performance, and a proposed future enhancement.

2 Data Handling

Raw text data is usually noisy and unstructured, containing various inconsistencies such as typos, slang, abbreviations and irrelevant information. Pre-processing removes the noise and irrelevant information, leading to a much richer feature extraction resulting in a better model performance ([GeeksforGeeks, 2025c](#)).

Upon analysis, the data had many spelling errors, such as 'emial' (email), 'wat' (what), and abbreviations such as 'acct' (account), '2' (to). According to the NLTK library ([Stack Overflow, 2015](#)), 24.5% of the words in the raw text were identified as misspelled (Figure 1), as shown below. Spell correction was initially attempted using open source libraries ([GeeksforGeeks, 2025b](#)), however these methods were unreliable, as they occasionally modified already correct words into incorrect ones. For example, in one instance the word '*notifications*' was incorrectly changed to '*modifications*', even though the original word was correct (Figure 1), as shown below. Since such changes could impact model performance, spell correction was not applied in the final pre-processing pipeline.

Calculating OOV words in training data

```
[91]: import nltk
      from nltk.corpus import words

      data = tokenize(tickets)
      data = lowercase(data)
      data = remove_punctuation(data)
      data = remove_hidden_characters(data)
      data = whitespace_remove(data)
      english_vocab = set(words.words())

      total_words_count = 0
      non_vocab_words_count = 0

      for sentence in data:
          for word in sentence:
              total_words_count += 1
              if word not in english_vocab:
                  non_vocab_words_count += 1

      percentage = round(non_vocab_words_count * 100 / total_words_count, 2)
      print(f"Total words: {total_words_count}")
      print(f"OOV words: {non_vocab_words_count}")
      print(f"Percentage of OOV words: {percentage}%")

      Total words: 2090
      OOV words: 512
      Percentage of OOV words: 24.5%
```

Spelling Mistakes Corrector not being reliable

'notifications' converted to 'modifications'

```
[83]: from textblob import TextBlob
      sample_sentence = df['Ticket Description'].iloc[229]
      print(sample_sentence)

      for word in sample_sentence.split():
          modified_word = str(TextBlob(word).correct())
          print(f"{word:15} -> {modified_word}")

      stop promotional notifications but keep important ones?
      stop -> stop
      promotional -> promotion
      notifications -> modifications
      but -> but
      keep -> keep
      important -> important
      ones? -> ones?
```

Figure 1: Percentage of OOV words and an example of spell corrector unreliability

For the first pipeline, pre-processing includes tokenisation (segments the text into tokens), lowercasing, punctuation removal, hidden character removal, whitespace removal, stopword removal and stemming. Stemming was used as a final step of pre-processing as it reduces the

word to their base or root form by removing prefixes and suffixes, making it easier to analyse and process text (GeeksforGeeks, 2025a).

In the feature engineering stage, a Label Similarity step was performed. For the first pipeline, if a sentence contained the target label, that single word was extracted and retained as the sole representation for the entire sentence. If the label wasn't present, the pre-processed sentence after stemming was kept as is. For this scenario, the approach was expected to perform well because the target labels appeared 245 times in the dataset out of 308 instances, suggesting that many sentences explicitly contain the label token as shown below in Figure 3. Figure 2 below summarises all the steps of the first pipeline.

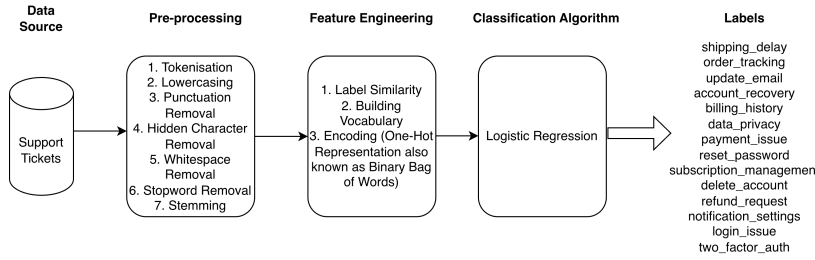


Figure 2: Steps of the first pipeline

```

tickets = df['Ticket Description'].tolist()
labels = df['Classification'].tolist()
data = tokenise(tickets)
data = lowercase(data)
count = 0
for sentence, label in zip(data, labels):
    found = False
    for word in sentence:
        if word in label and not found:
            count = count + 1
            found = True
print(count)
245

```

Figure 3: Count of target labels in tickets description

For the second pipeline, the pre-processing steps mirrored the first, with the notable exception of stemming. Stemming was deliberately omitted to preserve the original forms of the words, ensuring more meaningful semantic comparisons. To evaluate this semantic similarity, the pipeline leveraged static `word2vec` embeddings (Rehurek, 2024). Specifically, it utilised 300-dimensional vectors trained on a corpus of 100 billion words from Google News using the continuous bag-of-words (CBOW) architecture (Mikolov et al., 2013).

The Label Similarity step involved identifying the single word within a sentence that was most semantically related to the target label. To achieve this, the pipeline iterates through each candidate word in the sentence. If a word was missing from the `word2vec` vocabulary, spell correction was attempted using an open-source library¹. If an embedding remained unavailable after correction, the word vector was initialised randomly, adopting the same approach as Kim (2014).

Once the embeddings for both the candidate word ($\mathbf{e}_w$) and the target label ($\mathbf{e}_l$) are acquired, their semantic closeness is quantified using cosine similarity (Airbnb Engineering, 2018):

$$\text{Cosine Similarity}(\mathbf{e}_w, \mathbf{e}_l) = \frac{\mathbf{e}_w \cdot \mathbf{e}_l}{\|\mathbf{e}_w\| \|\mathbf{e}_l\|} \quad (1)$$

The representative word for the sentence, w^* , is then determined by evaluating all words w within the sentence S and selecting the one that has the highest overall cosine similarity score with the target label:

$$w^* = \arg \max_{w \in S} \left(\frac{\mathbf{e}_w \cdot \mathbf{e}_l}{\|\mathbf{e}_w\| \|\mathbf{e}_l\|} \right) \quad (2)$$

For both pipelines, before generating the one-hot feature representation (binary bag of words), the vocabulary was built using all the text available. An unknown token was also added in the vocabulary to account for any unknown word encountered during inference.

¹This is different from spell check run through open-source libraries as this checks in `word2vec` embeddings vocabulary.

Figure 4 below shows the word clouds for several labels generated from the second pipeline, capturing the most similar word to each label using `word2vec` embeddings and cosine similarity. On analysis, the pre-processing identifies semantically related terms such as ‘*acct*’ (an abbreviation) and ‘*remove*’ for `delete_account`, ‘*paypal*’ and ‘*card*’ for `payment_issue`, ‘*courier*’ and ‘*transit*’ for `shipping_delay`, and ‘*invoices*’ and ‘*duplicate*’ for `billing_history`.

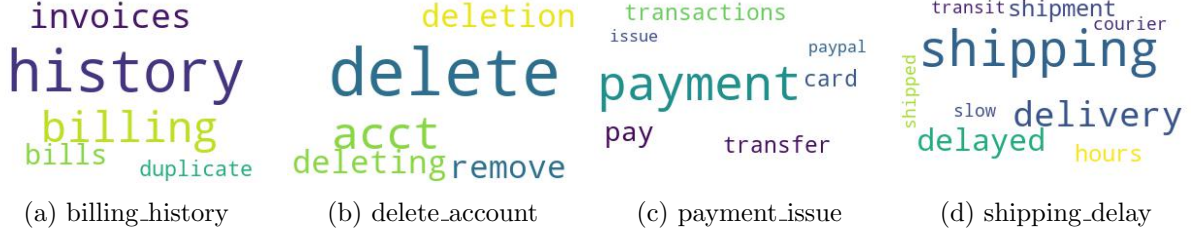


Figure 4: Word clouds for selected labels from the second pipeline

3 Data Modelling

Both pipelines model sentences using one-hot encoded word features (binary bag of words). There were 14 classes in the dataset for 308 data points, with many classes having 20-24 data points, and 2 classes with lesser data points (16 for `login_issue`, 10 for `two_factor_auth`), resulting in a slightly imbalanced dataset.

To mitigate this issue, the dataset was split into training and test sets using a stratified (Encord, 2024) 70:30 ratio to preserve the relative proportion of each class across the split. Logistic Regression was used for classification, as it demonstrated superior performance compared to other classification models such as Random Forest and SVMs. Hyperparameter tuning was performed on both pipelines by adjusting the regularisation strength to maximise performance.

4 Evaluation

Quantitatively, Pipeline 2 clearly outperformed Pipeline 1 across all evaluation metrics, despite using a much smaller vocabulary (Table 2), as it leverages `word2vec` embeddings, which provide a rich semantic representation. Pipeline 2 demonstrated almost no overfitting, as we see a negligible gap between training and test accuracy, which suggests the models generalises very well on unseen data. Table 1 below summarises all metrics for both pipelines.

Table 1: Comparison of performance of both pipelines

Metrics	Pipeline 1	Pipeline 2
Training Accuracy	98.6%	96.3%
Test Accuracy	92.5%	94.6%
Precision	94.5%	96.4%
Recall	91.8%	93.1%
F1 Score	92.4%	94%

Table 2: Vocabulary size used in each pipeline

	Pipeline 1	Pipeline 2
Vocabulary Size	150	60

Qualitatively, the second pipeline performed better than the first one because the reduced vocabulary resulted in a less sparse one-hot vector (binary bag of words). For the first pipeline, the model behaved as expected, evidenced by the confusion matrix (Figure 5) below. For the `two_factor_auth`, the model performed worst on unseen data, with an accuracy of 66.6%. The label just had just 7 examples in the training data, making it the least represented class. Underrepresented classes are more difficult to learn because models are not exposed to them as often (Collins et al., 2018). Overall as well, the dataset was slightly imbalanced, which increased the difficulty of text classification, and this is a known problem in machine learning as described by Chawla et al. (2004)

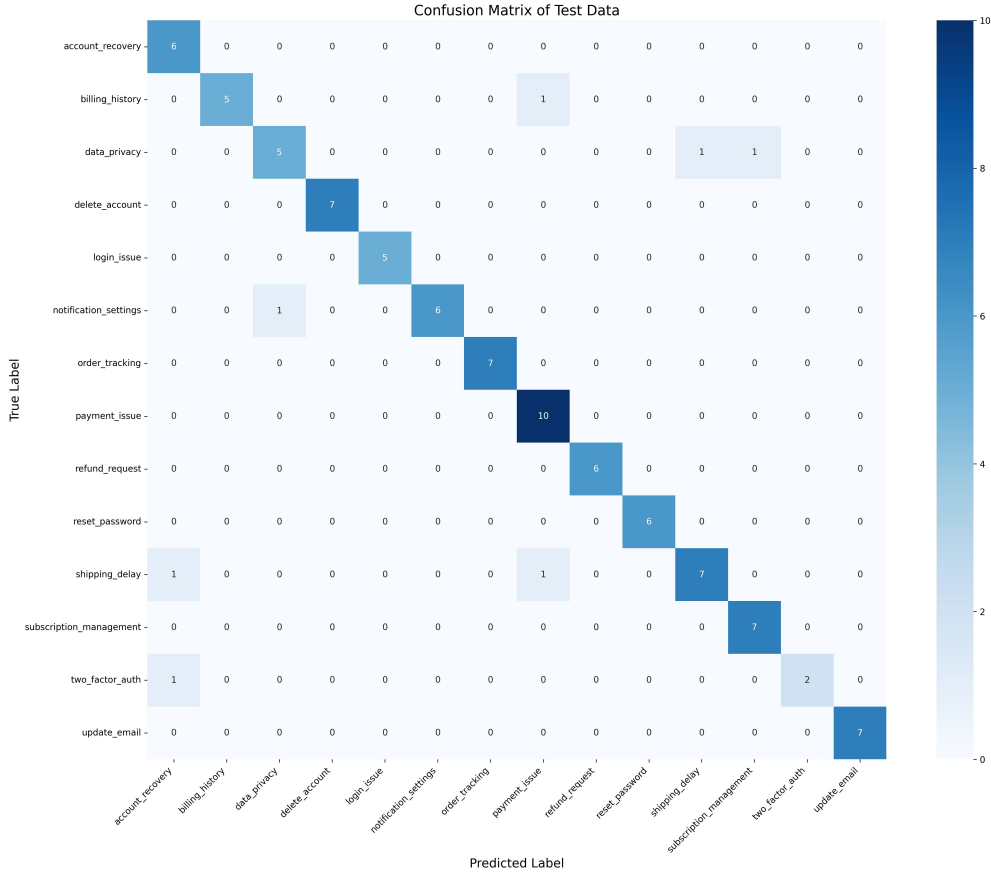


Figure 5: Confusion matrix for pipeline 1 on the test dataset

For the `shipping_delay` label also, a low accuracy of 77.7% was observed on test data. For this label, misclassified sentences contained words such as *‘claim’* and *‘received’*, which confused the model due to the semantic overlap between classes, making them difficult to separate (Japkowicz, 2000), as shown below in Figure 6.

```

DF index: 276
Sentence: package lost during shipment - claim process?
True: shipping_delay
Pred: payment_issue

DF index: 232
Sentence: shipment marked delivered but not received!
True: shipping_delay
Pred: account_recovery

```

Figure 6: Misclassified examples for the `shipping_delay` label

Also, the `shipping_delay` label contributed to the highest vocabulary of 40 tokens amongst all labels, which increased the sparsity making classification more complicated. Figure 7 below

summarises how much each label contributes to the vocabulary size for the first pipeline.

```
shipping_delay 40
account_recovery 31
two_factor_auth 22
data_privacy 18
login_issue 17
billing_history 16
subscription_management 14
payment_issue 13
update_email 12
notification_settings 9
order_tracking 7
delete_account 7
reset_password 2
refund_request 2
```

Figure 7: Label-wise vocabulary size distribution for pipeline 1

For the second pipeline, a similar pattern was observed. The two labels that achieved relatively lower accuracy compared to the overall model performance were `login_issue` and `two_factor_auth`, which were also the least represented classes in the training data.

Precision measures the model’s ability to correctly identify instances of a particular class, while recall measures its ability to find all such instances (Evidently AI, 2025). While both are key metrics, for this use case, it was important to maximise precision rather than recall (Data Science Stack Exchange, 2018), and the second pipeline successfully achieved this objective. Misclassifying instances of a particular class can lead to delays and inefficiencies in incident management. For example, if a ticket related to `login_issue` is incorrectly identified as `payment_issue`, re-routing the ticket to a different team can increase the turnaround time for resolution and significantly impact the customer experience. To evaluate the model fairly across all categories, macro-averaged precision and recall are used, as this approach assigns equal weight to each class regardless of its frequency (Evidently AI, 2025).

5 Conclusion

Through the present work, the second pipeline utilising `word2vec` performed remarkably well. This result suggests that this approach for text classification can be applied to other domains as well, where sentences are relatively short. In this dataset, the maximum sentence length was only nine words, indicating that embedding-based similarity methods can still perform effectively even with limited contextual information. The results obtained also add to the well-established evidence that unsupervised pre-training of word vectors is an important ingredient for NLP tasks (Kim, 2014).

The first pipeline, albeit based on a more conventional approach, is also suitable for this use case as it leverages the fact that the target labels frequently appear within the sentences. Despite its simplicity, it produces strong results. The choice between the two approaches ultimately depends on the scale and requirements of the application for which support tickets needs to be classified. For smaller business applications, the first approach may be preferable due to its significantly lower memory requirements compared to the `word2vec`-based pipeline. However, if the system scales and becomes more critical to business operations, the second approach may be more suitable as it can capture semantic relationships more effectively.

An enhancement to the current approach would be to use data augmentation since the data was limited. The most natural form of data augmentation would be to replace words or phrases with their synonyms as described by Zhang et al. (2015), which in some cases has shown to improve model performance.

References

- Airbnb Engineering (2018). *Listing Embeddings for Similar Listing Recommendations and Real-Time Personalization in Search*. Available from: <https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e> [Accessed 8 March 2026].
- Chawla, N. V., Japkowicz, N., and Kolcz, A. (2004). *Editorial: Special Issue on Learning from Imbalanced Data Sets*. *ACM SIGKDD Explorations Newsletter*, 6(1):1–6. Available from: <https://doi.org/10.1145/1007730.1007733>.
- Collins, E., Rozanov, N., and Zhang, B. (2018). *Evolutionary Data Measures: Understanding the Difficulty of Text Classification Tasks*. *arXiv preprint*. Available from: <https://arxiv.org/abs/1811.01910>.
- Data Science Stack Exchange (2018). *When is precision more important over recall?* Available from: <https://datascience.stackexchange.com/questions/30881/when-is-precision-more-important-over-recall> [Accessed 8 March 2026].
- Encord (2024). *Train, Validation, Test Split for Machine Learning Datasets*. Available from: <https://encord.com/blog/train-val-test-split/> [Accessed 8 March 2026].
- Evidently AI (2025). *Accuracy, Precision and Recall in Multi-Class Classification*. Available from: <https://www.evidentlyai.com/classification-metrics/multi-class-metrics> [Accessed 8 March 2026].
- GeeksforGeeks (2025a). *Introduction to Stemming*. Available from: <https://www.geeksforgeeks.org/machine-learning/introduction-to-stemming/> [Accessed 8 March 2026].
- GeeksforGeeks (2025b). *Python TextBlob.correct() method*. Available from: <https://www.geeksforgeeks.org/python/python-textblob-correct-method/> [Accessed 8 March 2026].
- GeeksforGeeks (2025c). *Text Preprocessing in NLP*. Available from: <https://www.geeksforgeeks.org/nlp/text-preprocessing-for-nlp-tasks/> [Accessed 8 March 2026].
- Japkowicz, N. (2000). *The Class Imbalance Problem: Significance and Strategies*. In *Proceedings of the International Conference on Artificial Intelligence*, pages 111–117.
- Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. *arXiv preprint*. Available from: <https://arxiv.org/abs/1408.5882>.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., and Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. *arXiv preprint*. Available from: <https://arxiv.org/abs/1310.4546>.
- Rehurek, R. (2024). *Word2vec embeddings*. Available from: <https://radimrehurek.com/gensim/models/word2vec.html> [Accessed 8 March 2026].
- Stack Overflow (2015). *Is There a Corpus of English Words in NLTK?* Available from: <https://stackoverflow.com/questions/28339622/is-there-a-corpus-of-english-words-in-nltk> [Accessed 8 March 2026].

- Uber AI (2018). *COTA: Improving Uber Customer Care with NLP and Machine Learning*. Available from: <https://www.uber.com/en-GB/blog/cota/> [Accessed 8 March 2026].
- Zhang, X., Zhao, J., and LeCun, Y. (2015). *Character-level Convolutional Networks for Text Classification*. *arXiv preprint*. Available from: <https://arxiv.org/abs/1509.01626>.